VOLUME III · COMPLETE NOTE COVERAGE · PAGE BY PAGE

CMS 702 — Everything Left in the Lecture Notes

The gap-closer. Volumes I and II cover the syllabus and drill it; this volume walks the lecturer's 16 handwritten pages in order and picks up every definition, worked example and class assignment the first two did not reproduce — including the two the notes themselves left unresolved.

Prepared by **Mbosinwa Awunor** · www.mbosinwa.dev

Exam: Monday 03 Aug 2026 **Time:** 11:00 – 14:00 **Venue:** the exam hall **Lecturer:** the lecturer **Units:** 3

1 Coverage map — every page of the notes, and where it is answered

Read this table once. If a row says **Vol III**, the material is in this document and you have not seen it in the other two.

NOTES §	PAGE	TOPIC	COVERED IN	WHAT IS HERE IN VOLUME III
1	1–2	Algorithms · the ten types	**Vol III**	All ten types **defined**, not just named — §2
2	2–3	Asymptotic analysis · notations · upper & average bounds	Vol I §3	Bound wording restated in §3
3	3	The five complexity classes	Vol I §3	—
4	4	Sorting and searching	Vol I §4	—
5	5–6	Hashing · components · worked example · collision · load factor	Vol I §5	**The page-5 assignment answered** — §4
6	6	Dynamic programming	Vol I §4	—
7	6–7	Data structures · types · operations · arrays · linked lists	Vol I §1, §8	—
8	8	Stacks	Vol I §8	—
9	8	Queues	Vol I §8	—
10	8–11	Trees · terminology · node · height, depth, degree	Vol I §6	—
11	11–12	Types of trees · the perfect-tree formulas	Vol I §6	The complete-binary-tree example from p.12 — §7
12	12	Binary search trees · properties · worked build	Vol I §7	—
13	13	In-order successor · Examples 1 and 2 · the assignment	**Vol III**	**Both examples worked** — §5
14	13–14	In-order predecessor	Vol I §7	—
15	14	BST traversal · the nine-letter class example	**Vol III**	**The tree reconstructed and verified** — §6
16	15–16	Construction from traversals · pre + in · **pre + post**	**Vol III**	**The pre-order + post-order method and worked example** — §7

VERDICT ON COVERAGE

With this volume, **every section, worked example and assignment in the 16 handwritten pages is accounted for**, plus the four outline topics that never reached the notes at all (graphs, run-time storage management, numerical algorithms, string processing) which sit in Volume I §9 and §10. Nothing in the course material is now unrepresented.

2 The ten types of algorithm — defined, not just listed

Volume I lists these for recall. The notes **define** six of them, and a question worded "*list and explain any five types of algorithm*" needs the explanations. One sentence each is enough.

#	TYPE	DEFINITION TO WRITE	EXAMPLE
1	**Brute force**	Tries **all possible solutions** to a specific problem until the correct one is found.	Linear search; trying every password combination
2	**Recursive**	A method that breaks a problem into smaller sub-problems and repeatedly breaks the problem down until it is able to solve it.	Factorial, tree traversal, Towers of Hanoi
3	**Encryption**	Utilises cryptographic techniques to transform data into a secure form, ensuring confidentiality and privacy in digital communication.	AES, RSA
4	**Backtracking**	Uses **trial-and-error** techniques to explore potential solutions, abandoning a path as soon as it cannot lead to a valid solution.	N-queens, sudoku solving, maze routing
5	**Search**	Designed to find a specific target within a dataset, enabling effective retrieval of information.	Sequential search, binary search
6	**Sort**	Aims at arranging the elements of a list in a specific order — ascending, descending or lexicographic.	Merge, quick, heap sort
7	**Divide and conquer**	Divides the problem into **independent** sub-problems, solves each one, and combines their solutions into the answer.	Merge sort, quick sort, binary search
8	**Greedy**	Makes the choice that looks best **at each step**, never reconsidering, in the hope that the local optima give the global optimum.	Coin change, Dijkstra, Huffman coding
9	**Dynamic programming**	Solves a complex problem by breaking it into smaller sub-problems, solving each **once** and storing the result for reuse.	Fibonacci, knapsack, longest common subsequence
10	**Randomized**	Uses a **random choice** at some point in its logic, so its behaviour or running time depends partly on chance.	Randomized quick sort (random pivot)

The contrast most likely to earn a mark: divide and conquer splits a problem into *independent* sub-problems and combines their answers; **dynamic programming** applies where the sub-problems *overlap*, and it stores each answer so it is computed only once. **Greedy** differs from both — it never revisits a decision, which makes it fast but not always optimal.

3 Upper and average complexity bound — the notes' own wording

UPPER COMPLEXITY BOUND

Provides a **guarantee of the algorithm's performance in the worst-case scenario**. It ensures the algorithm will not perform unexpectedly poorly under any circumstances. This is why Big O — the upper bound — is the notation used by default when an algorithm is analysed.

AVERAGE COMPLEXITY

Considers the **expected performance** of an algorithm, given that all possible inputs of a certain size assume a certain **distribution** of input. It is more representative of everyday behaviour than the worst case, but it says nothing about the guarantee.

If a question asks you to "distinguish between the upper bound and the average complexity bound", the answer is exactly this pair: one is a **guarantee under the worst input**, the other is an **expectation over a distribution of inputs**. Add that the worst case is stated with **O** and the exact average behaviour with **Θ**.

4 Page-5 assignment — the array as a map

As set in class: "*Consider an array as a map where the key is the index and the value is the value at that index. Find the value at* `A[i]`*, where* `A` *is the location and* `i` *is the integer.*"

Answer. An array is the simplest possible hash-style map: the **key is the index** `i`, the **value is the element stored there**, and the "hash function" is the **identity function** — the key *is* the slot number, so no transformation and no collision is possible.

The value at `A[i]` is found by address arithmetic, not by searching:

```
address of A[i] = base address of A + ( i × size of one element )
```

Because the machine computes that address in a fixed number of steps regardless of how large the array is, the lookup is **O(1) — constant time**. This is precisely the property hashing tries to buy for arbitrary keys such as strings: the hash function converts a non-numeric key into an index so that it too can be reached in one step.

```
A  =  Int Array(10)      base address = 1000
      element size = 4 bytes

 key (index):  0    1    2    3    4  …
 value:      [35] [33] [42] [10] [14] …
 address:    1000 1004 1008 1012 1016

A[3]  →  1000 + (3 × 4)  =  1012  →  value 10
```

The sentence that finishes the answer: an array is a map whose keys are restricted to consecutive integers; a **hash table** generalises it by allowing *any* key — a string or an arbitrary integer — which a hash function maps down onto those same array indices.

5 In-order successor — the two class examples

Volume I gives the method; these are the exact two examples from page 13, which are small enough to be reproduced verbatim in an answer.

EXAMPLE 1

```
root = [2, 1, 3],  K = 2

      2
     / \
    1   3
```

In-order = `1, 2, 3`. The node after 2 is **3**.

> ∴ **successor of K = 3**
> Structural reading: K has a right sub-tree, so the successor is the **smallest node in that right sub-tree**.

EXAMPLE 2

```
root = [3, 2, 1],  K = 3

      3
     /
    2
   /
  1
```

In-order = `1, 2, 3`. Nothing follows 3 — it is the largest value in the tree.

> ∴ **successor of K = −1**
> The convention in the notes: **−1** is returned when no in-order successor exists.

THE PAGE-13 ASSIGNMENT

```
root = [20, 8, 22, 4, 12,
        N, N, N, N, 10, 14]
K = 8

           20
          /  \
         8    22
        / \
       4   12
          /  \
         10   14
```

In-order = `4, 8, 10, 12, 14, 20, 22`.

> ∴ **answer = 10** — worked in class.

HOW TO READ THE LEVEL-ORDER ARRAY NOTATION

A list like `[20, 8, 22, 4, 12, N, N, N, N, 10, 14]` is a **level-order** listing: read the tree row by row, left to right, writing **N** for an absent child. Level 0 is `20`; level 1 is `8, 22`; level 2 is `4, 12` then `N, N` for 22's two missing children; level 3 gives 4's two missing children `N, N` and then 12's children `10, 14`. Rebuild the picture before answering anything — never try to reason from the list itself.

6 Page-14 traversal example — the tree recovered

The notes record three traversal sequences from the board but flag the tree diagram as too faint to read. The tree is fully recoverable from any two of those sequences, and it is worth having, because these letter sequences are exactly the kind of thing that reappears on a paper.

WHAT THE NOTES RECORDED

```
In-order    = B, D, A, G, E, C, H, F, I
Pre-order   = A, B, D, C, E, G, F, H, I
Post-order  = D, B, G, E, H, I, F, C, A
```

RECONSTRUCTION FROM PRE-ORDER + IN-ORDER

1. First of the pre-order is the root → **A**. In the in-order, `B D` lies to its left and `G E C H F I` to its right.
2. Left sub-tree: pre `B D`, in `B D` → root **B**; D is *after* B in the in-order, so D is B's **right** child.
3. Right sub-tree: pre `C E G F H I`, in `G E C H F I` → root **C**; `G E` left, `H F I` right.
4. Under C, left: pre `E G`, in `G E` → root **E** with G as its **left** child. Right: pre `F H I`, in `H F I` → root **F** with H left and I right.

THE TREE

```
          A
        /   \
       B     C
        \   /  \
         D E    F
           /   / \
          G   H   I
```

Verification — all three sequences must match, and they do:

In-order (L-Root-R)	`B, D, A, G, E, C, H, F, I` ✓
Pre-order (Root-L-R)	`A, B, D, C, E, G, F, H, I` ✓
Post-order (L-R-Root)	`D, B, G, E, H, I, F, C, A` ✓

Note that this tree is **not** a binary search tree — its in-order traversal is not alphabetical. It is an ordinary binary tree, which is why the traversals must be read off the structure rather than guessed from the ordering.

7 Construction from pre-order + post-order — the page-16 method

Volume I covers **pre + in** and **post + in**. The notes also work a third combination, **pre-order + post-order**, which behaves differently and has a caveat worth a mark.

The method

1. The **first** element of the pre-order and the **last** element of the post-order are the same node — that is the **root**.
2. Take the **second** element of the pre-order. That node is the **root of the left sub-tree**.
3. Find that node in the **post-order**. Everything up to and including it forms the **left sub-tree**; everything after it, excluding the overall root, forms the **right sub-tree**.
4. Split the pre-order the same way by size, and **recurse** on each side.

WORKED EXAMPLE FROM THE NOTES

```
Pre-order  = F, B, A, D, C, E, G, I, H   [Root,L,R]
Post-order = A, C, E, D, B, H, I, G, F   [L,R,Root]
```

1. Root = first of pre = last of post = **F**.
2. Second of pre = **B** → root of the left sub-tree. In the post-order, everything up to and including B is `A, C, E, D, B` → the left sub-tree; the remainder `H, I, G` → the right sub-tree.
3. Left: pre `B A D C E`, post `A C E D B` → root **B**; its left child is **A** (next in pre, and A ends the left block in post), its right child is **D**, whose children are **C** and **E**.
4. Right: pre `G I H`, post `H I G` → root **G**; next in pre is **I**, whose child is **H**.

Verification: pre-order of the drawn tree = `F, B, A, D, C, E, G, I, H` ✓ · post-order = `A, C, E, D, B, H, I, G, F` ✓ · and its in-order comes out as `A, B, C, D, E, F, G, H, I` — alphabetical, so this one **is** a valid BST.

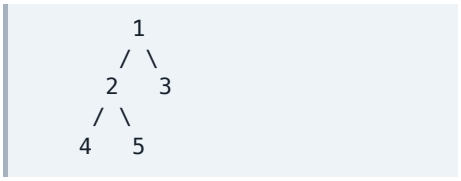
The caveat worth a mark. Pre-order + post-order does **not** always give a unique tree. When a node has only **one** child, the pair cannot tell you whether that child is the left or the right one — here, G has the single child I, and the reconstruction assumes it is the right child. The pair is unique only for a **full binary tree** (every node with 0 or 2 children). **In-order paired with either of the other two is always unique**, which is why in-order is the sequence you actually need.

The three combinations side by side

GIVEN	WHERE THE ROOT IS	HOW TO SPLIT	UNIQUE?
Pre-order + in-order	**First** of the pre-order	Find the root in the in-order; left of it is the left sub-tree, right of it the right	**Always unique**
Post-order + in-order	**Last** of the post-order	Identical split on the in-order	**Always unique**
Pre-order + post-order	First of the pre = last of the post	Second of the pre is the left sub-tree's root; locate it in the post-order and cut there	Only for a **full** binary tree

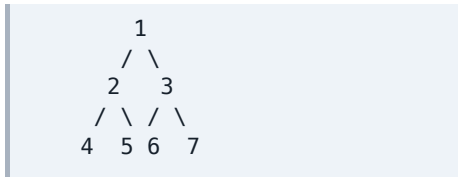
8 The tree-type diagrams from pages 11–12

FULL BINARY TREE



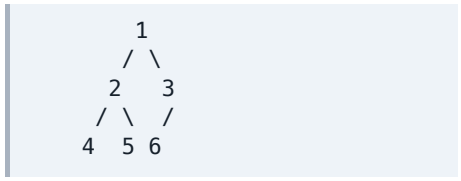
Every node has **0 or 2** children. Node 3 is a leaf (0 children); nodes 1 and 2 have two each. No node has exactly one child.

PERFECT BINARY TREE



Every internal node has exactly 2 children **and** all leaves are on the same level. Here h = 2, so leaves `l = 2² = 4` and nodes `n = 2³ − 1 = 7` ✓

COMPLETE BINARY TREE P.12



Every level filled except possibly the last, whose leaves **lean left** — node 3 has a left child but no right sibling for it. This is the exact diagram from the notes.

THE DISTINCTION IN ONE LINE EACH

Full — no node has exactly one child. **Perfect** — full *and* every leaf on the same level. **Complete** — every level filled except the last, which fills from the left. **Every perfect tree is both full and complete; the converse fails in both directions** — the complete tree above is not full (node 3 has one child), and a full tree can be lopsided and so not complete.

9 One-page note map — the whole course in a grid

TOPIC	THE SINGLE LINE YOU MUST BE ABLE TO WRITE	THE NUMBER TO QUOTE
Algorithm	Step-by-step procedure / set of instructions to solve a specific problem	10 types
Asymptotic analysis	Running time as a mathematical function f(n); describes limiting behaviour	3 cases, 3 notations
Notations	O upper/worst · Θ tight/both · Ω lower/best, all for $n \geq n_0$	—
Complexity classes	Constant, logarithmic, linear, quadratic, exponential	$O(1) < O(\log n) < O(n) < O(n^2) < O(2^n)$
Sorting	Arranging list elements in a specific order	5 named: merge, quick, bucket, heap, counting
Searching	Finding a target within a dataset	Sequential O(n) · binary O(log n), sorted only
Hashing	Lookup — mapping arbitrary data to a tabular index via a hash function	3 components · 4 properties · 2 collision fixes · O(1)
Load factor	Items in the table ÷ size of the table	7/10 = 0.7
Dynamic programming	Breaking a complex problem into smaller sub-problems and reusing their results	Fibonacci $O(2^n) \rightarrow O(n)$
Data structure	Organization, management and storage format enabling efficient access and modification	3 categories · 6 operations
Arrays	Fixed-size cannot be altered, indexes numbered; dynamic can be resized	A[i] found in O(1) by address arithmetic
Linked lists	Nodes holding data plus a pointer to the next node	3 types: singly, doubly, circular
Stack	ADT implementing LIFO, open at one end, PUSH and POP	1 pointer (top)
Queue	ADT implementing FIFO, open at both ends, Enqueue() and Dequeue()	2 pointers (front, rear)
Tree	Hierarchical non-linear structure of nodes connected by edges	3 properties · leaf height 0

TOPIC	THE SINGLE LINE YOU MUST BE ABLE TO WRITE	THE NUMBER TO QUOTE
Height / depth	Height counts up from the deepest leaf; depth counts down from the root	Perfect tree: $l = 2^h$, $n = 2^{(h+1)} - 1$
BST	Left node less than parent, right node greater than parent	4 properties · in-order is sorted
Traversals	In L-Root-R · Pre Root-L-R · Post L-R-Root	Missing successor -1 · missing predecessor null
Reconstruction	Pre gives the root at the front, post at the back; in-order tells you where to split	Pre + post unique only for a full tree
Graph	$G = (V, E)$ — vertices joined by edges	Matrix $O(V^2)$ · list $O(V + E)$ · BFS/DFS $O(V + E)$